Identity-Document Fraud Detection with a Fine-Tuned DINOv2 and Data-Grounded Synthetic Tampering

FREUID Challenge 2026 (IJCAI-ECAI) — Technical Report Team: **hyunsooochung** · Code: Apache-2.0 · Submitted model: `synth_recapture_v1` (epoch 11)

1. Introduction

The task is binary fraud detection on identity-document images: emit a continuous fraud score `P(fraud) ∈ [0,1]` (`1 = fraud`). The primary metric is **AuDET** (area under the DET curve; lower is better), a pure **rank** metric — only score ordering matters. The challenge is deliberately hard: the hidden test emphasizes **print-and-capture ("analog hole") attacks** and **document types not seen in training**, whereas the released training set is almost entirely *digital*. A detector that leans on digital forensic noise scores near-zero on an in-domain split but collapses on the real test, since reprinting erases that noise. Our approach therefore pairs a strong, fully-adapted self-supervised backbone with a **data-grounded synthetic-fraud augmentation** and an **analog-recapture augmentation** aimed squarely at the print-capture threat.

2. Method

2.1 Architecture

A **DINOv2 ViT-B/14** backbone (`vit_base_patch14_dinov2.lvd142m`), **fully fine-tuned**, with an **attention-pooling** head over the patch tokens and a single fraud logit (zero-init). Full fine-tuning of a strong SSL backbone was by far the biggest lever in our experiments (a properly-tuned DINOv2 moved AuDET from ~0.21 with an EfficientNet-V2 baseline to ~0.07); a *frozen* backbone with a light head was much worse, so feature adaptation — not head complexity — is what matters.

2.2 `synth_tamper` — data-grounded synthetic fraud

Genuine cards vastly outnumber frauds in training, so the model never sees enough forgeries. We manufacture them on the fly: each epoch a fraction (`prob = 0.3`) of bona-fide cards is turned into a synthetic fraud (label 1), reproducing tells found in a **100-image manual analysis** of the real frauds: - **Face-paste (≈80% of tampers):** almost every real fraud has a pasted portrait — a sharp face with a hard rectangular seam over which the card's guilloche/overlay does not continue. We paste a donor face (per-document-type pool, validation ids excluded to prevent leakage), plus a document-specific *colour-face-on-grayscale-body* variant. - **Field-carve (≈20%, text):** a bounded value field (e.g. date of birth) is rewritten so the digits stay readable but the background micro-texture is destroyed — calibrated pixel-by-pixel against real examples.

2.3 Analog-recapture augmentation

To target the challenge's print-and-capture attack, a fraction (`recapture_prob = 0.5`) of all training images additionally receive a **calibrated analog degradation** (softening + illumination gradient + desaturation + recompression), **label-preserving**, on top of any tampering. This teaches invariance to the print→capture channel that erases digital cues.

2.4 Training

`BCEWithLogits` + a **pairwise soft-AUC** term (weight 0.1), matched to the rank metric. AdamW with **layer-wise LR decay 0.7**, 2-epoch warmup, **cosine annealing over 20 epochs**, base `lr = 5e-5`, weight decay 0.05, mixed precision (AMP), seed 42. The validation split is stratified by (label × document type); the per-epoch compass is an **analog "recapture probe"** (a deterministic recaptured copy of the held-out split) — chosen because it does not saturate the way an in-domain probe does.

3. Data (including external sources)

- **Training data:** the official **FREUID** training set only — 69,352 labelled identity- document images (`train_labels.csv`). No other datasets were used. The data is proprietary to the challenge and is not redistributed in our repository.
- **External pre-trained model: DINOv2 ViT-B/14** self-supervised weights (`vit_base_patch14_dinov2.lvd142m`, via `timm`; Apache-2.0). These are the only external weights; they are fine-tuned end-to-end and are fully contained in our checkpoint (so inference needs no download — see §5).
- No external labelled fraud data, no presentation-attack or forgery-localization networks.

4. Inference

Multi-scale **test-time augmentation** at `[476, 518, 560]`, **rank-averaged** across scales (no horizontal flip — documents carry orientation; rank-averaging suits the rank metric). The submitted model is **epoch 11** of the run (`synth_recapture_v1_ep11`). The fraud score is the rank-averaged sigmoid output, higher = more fraudulent. The reproducibility container (`docker/prepare_submission.py`) reads a flat directory of test images and writes one `id,label` row per image, reusing this exact inference path so its predictions match our Kaggle submission.

5. Results

The submitted `synth_recapture_v1` (epoch 11) reaches public **AuDET ≈ 0.026**. Ablations on the public leaderboard confirm the `synth_tamper` augmentation is the decisive lever — adding it to an otherwise-fixed recipe improved AuDET from **0.039** → **~0.016** (the first added feature that helped) — while backbone choice and head complexity were secondary. Full fine-tuning of DINOv2 with the ranking-aware loss and multi-scale TTA is the backbone of every strong result we obtained.

6. Reproducibility (commands, Docker image, hardware)

Hardware: a single **NVIDIA A100 (80 GB)**, mixed precision; ~10 min/epoch. Inference runs on GPU or CPU (auto-selected; checkpoint loads with `map_location="cpu"`).

Train (reproduces the checkpoint):

```
python -m freuid.train --config configs/synth_recapture_v1.yaml   # -> checkpoints/synth_reca
```

Offline Docker (organizer sandbox contract — no network, all weights embedded):

```
cp checkpoints/synth_recapture_v1_ep11.pt docker/model/synth_recapture_v1_ep11.pt
docker build -t freuid-repro:local .
docker run --network none \
    -v /path/to/flat/test/images:/data:ro \
    -v "$(pwd)/out:/submissions" \
    freuid-repro:local
# -> out/submission.csv   (id,label ; label = fraud score, higher = more fraud; one row per i
```

Full details — environment, data layout, and the plain (non-Docker) inference command — are in `REPRODUCE.md`. The fine-tuned checkpoint carries all weights, so the backbone is built with `pretrained=False` and nothing is downloaded at run time. GPU + AMP are not bit-deterministic, so a re-run reproduces the score *band*; the shipped checkpoint reproduces the exact ranked predictions.